

Big Data: Foundations, Architecture, and Systems



What "Big Data" Actually Denotes

Big data is not simply "a lot of data." The term denotes datasets and workloads whose **volume, velocity, or variety** exceed the capacity of conventional single-machine RDBMS architectures to capture, curate, manage, and process within tolerable elapsed time.

Doug Laney (META Group, 2001) first framed the problem as three dimensions of data management strain — **Volume, Velocity, Variety** — later popularized as the "3Vs" once Gartner adopted the term in 2011.

Operationally, treat big data as a **systems problem**: when a single machine's CPU, RAM, disk I/O, or network bandwidth becomes the bottleneck, the only durable solution is horizontal partitioning of data and computation across many commodity machines — which is the architectural thread running through this entire course.

REFERENCE POINT

2.5

quintillion bytes of data generated per day (IBM Marketing Cloud, 2017 estimate) — ~90% of all data in existence at the time had been created in the preceding two years.

Laney, D. (2001). "3D Data Management: Controlling Data Volume, Velocity and Variety." META Group Research Note.

The V-Framework, Extended

1	Volume	Scale of data at rest — terabytes to exabytes. Forces a shift from scale-up (bigger machine) to scale-out (more machines) architectures.
2	Velocity	Rate of data generation and required processing latency — from batch (hours) to near-real-time (seconds) to streaming (milliseconds).
3	Variety	Heterogeneity of structure: structured (relational), semi-structured (JSON, XML, logs), and unstructured (text, images, video).
4	Veracity	Uncertainty and noise in data — added by IBM (2012); provenance, missing values, and inconsistency directly affect model and query validity.
5	Value	Economic yield extracted per unit of storage/compute cost — added by Oracle/industry usage; the justification for the engineering overhead of the other four.

Why the Relational Model Alone Does Not Scale

Stonebraker and Çetintemel's 2005 ICDE paper "*One Size Fits All: An Idea Whose Time Has Come and Gone*" argues that general-purpose RDBMS engines optimized for OLTP carry architectural overhead — write-ahead logging, locking, buffer management, SQL processing — that becomes counter-productive for analytic, scientific, and stream workloads at scale.

Scale-up (vertical)

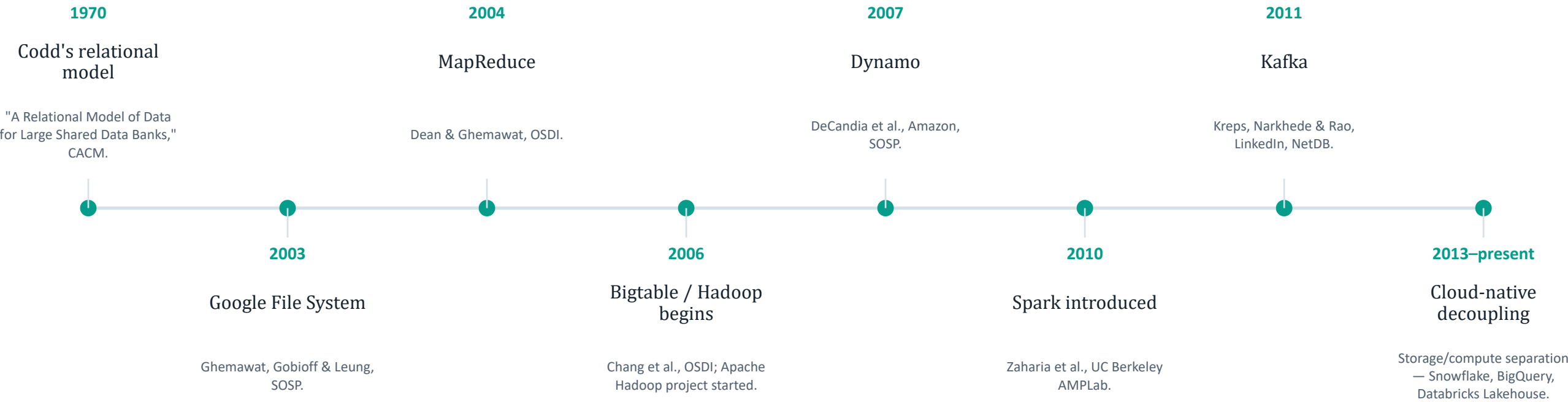
Add CPU, RAM, and faster disks to one machine. Bounded by the largest commercially available server; cost grows super-linearly; single point of failure remains.

Scale-out (horizontal)

Partition data and computation across many commodity machines. Near-linear cost scaling; failure becomes routine and must be engineered for, not avoided.

Big-data systems are, almost without exception, an engineering commitment to horizontal scaling and a corresponding acceptance of partial failure as a normal operating condition.

From the Relational Model to the Cloud-Native Stack



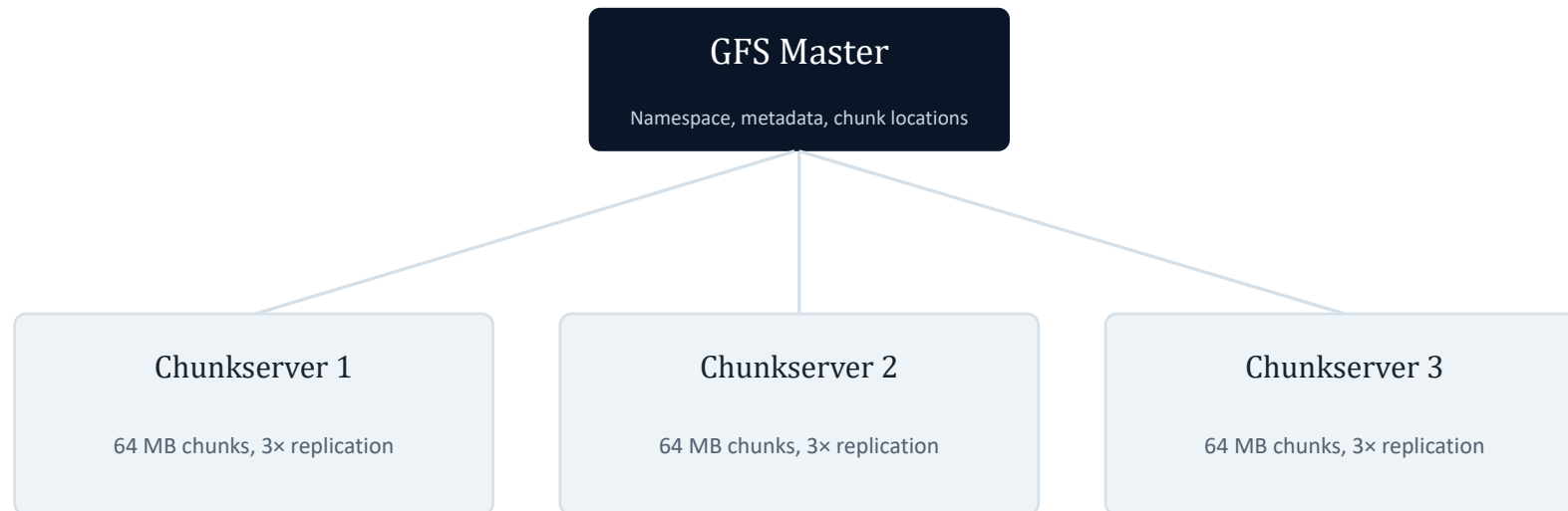
Distributed Storage & Batch Processing

Google's file system and the Hadoop implementation.



The Google File System (GFS)

Ghemawat, Gobioff, and Leung (SOSP 2003) designed GFS around three observed conditions at Google's scale: **component failure is the norm rather than the exception; files are huge (multi-GB); and most writes are appends, not overwrites.**



Design decisions: a single master simplifies coordination (metadata kept in memory); large 64 MB chunk size amortizes metadata and network overhead; and each chunk is replicated (default 3x) across chunkservers for fault tolerance without RAID.

MapReduce: A Model for Distributed Computation

Dean and Ghemawat (OSDI 2004) proposed a restricted programming model — two user-defined functions, **map** and **reduce** — under which the runtime automatically handles parallelization, data distribution, and fault tolerance across a cluster of commodity machines.

```
map(k1, v1) → list(k2, v2)
reduce(k2, list(v2)) → list(v3)
```

Between the two phases the runtime performs a **shuffle-and-sort**: intermediate key-value pairs are partitioned by key (typically $\text{hash}(\text{key}) \bmod R$) and routed to the reducer responsible for that partition.

Dean, J., & Ghemawat, S. (2004). "MapReduce: Simplified Data Processing on Large Clusters." Proc. 6th OSDI.

WORKED EXAMPLE — WORD COUNT

```
map(String key, String value):
    // key: document name
    // value: document contents
    for each word w in value:
        EmitIntermediate(w, "1");

reduce(String key, Iterator values):
    // key: a word; values: counts
    int result = 0;
    for each v in values:
        result += ParseInt(v);
    Emit(AsString(result));
```

This exact pseudocode appears in the original 2004 paper as the canonical illustration of the model.

MapReduce Execution and Fault Tolerance

- Master-worker coordination

One master assigns M map tasks and R reduce tasks to idle workers; it tracks the state (idle, in-progress, completed) of every task and the location of intermediate files.
- Locality-aware scheduling

The master schedules map tasks on (or near) the machine holding the input chunk in GFS, so most input is read from local disk rather than over the network.
- Worker failure

Detected via periodic pings. All map tasks the failed worker completed are re-executed (their output was local and is now unreachable), even if already "done."
- Straggler mitigation

Near the end of a phase, the master schedules backup executions of remaining in-progress tasks; whichever copy finishes first wins — addressing skew from slow hardware.
- Combiners

An optional local mini-reduce run on the mapper's output before shuffle, reducing network traffic for associative/commutative operations such as counting.

The Hadoop Ecosystem

Apache Hadoop (started 2006 by Doug Cutting and Mike Cafarella, originally for the Nutch search-engine crawler) is an open-source re-implementation of Google's GFS and MapReduce papers, later joined by a resource-management layer.

HDFS

Storage layer — open-source GFS analogue

YARN

Cluster resource manager (introduced Hadoop 2.0, 2013), decoupling scheduling from MapReduce

MapReduce

Original batch-processing engine, now one of several execution engines atop YARN

Ecosystem projects built on this foundation:

Hive

SQL-like query layer (HiveQL) compiling to MapReduce/Tez jobs; developed at Facebook.

Pig

Dataflow scripting language (Pig Latin) for ad hoc analysis pipelines; developed at Yahoo!.

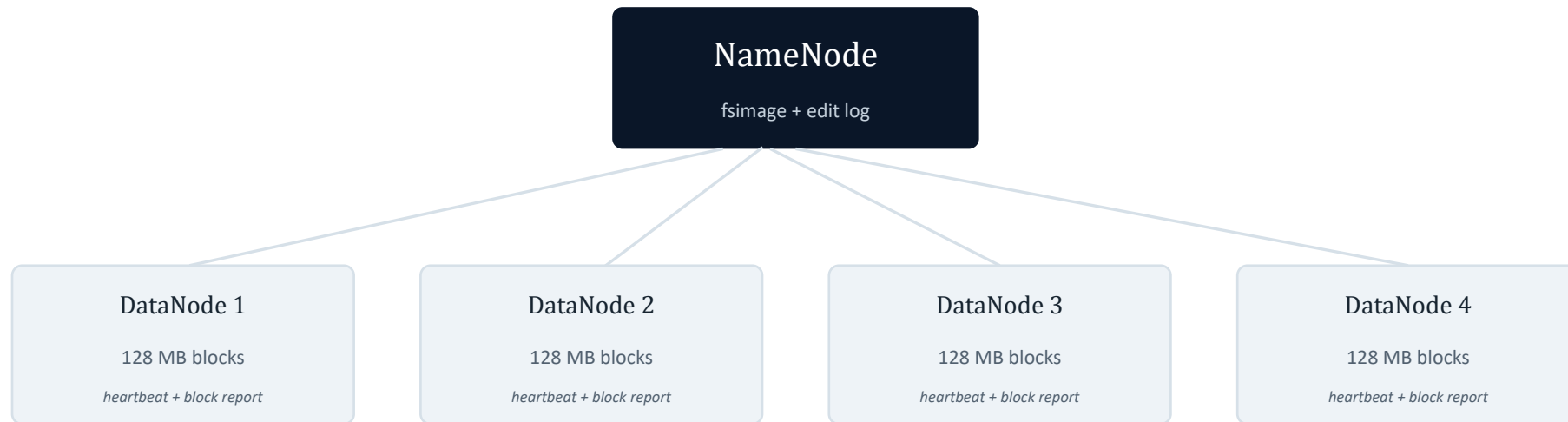
HBase

Column-family store modeled directly on Google's Bigtable, running atop HDFS.

ZooKeeper

Coordination service for distributed locking, configuration, and leader election.

HDFS: NameNode, DataNodes, and Replication



NameNode:

Holds the filesystem namespace and block-to-DataNode mapping in memory; single logical point of metadata authority (HA pairs and Secondary NameNode checkpoint against loss).

Block size:

Default 128 MB (64 MB in early Hadoop) — large relative to typical filesystems, minimizing seek-to-transfer time ratio for sequential scans of huge files.

Replication placement:

Default factor 3, rack-aware: one replica on the writer's node, one on a different rack, one on the same remote rack — balancing durability against cross-rack bandwidth cost.

Where the MapReduce Model Breaks Down

Materialization to disk between stages

Every job writes intermediate output to HDFS; a multi-stage pipeline or iterative algorithm re-reads and re-writes the full dataset at every step.

Poor fit for iterative workloads

Machine-learning training (gradient descent), graph algorithms (PageRank), and interactive queries revisit the same data across many passes — each pass re-pays full I/O cost.

Rigid two-stage DAG

Complex pipelines require chaining multiple MapReduce jobs, each with full job-scheduling and shuffle overhead, rather than expressing one flexible directed acyclic graph.

No native support for streaming or interactive queries

The model assumes finite, static input and batch-only execution; latency-sensitive and unbounded-stream workloads require different abstractions entirely.

These four limitations are the explicit motivation Zaharia et al. (2010, 2012) cite for the RDD abstraction underlying Apache Spark

In-Memory Computing & Stream Processing

Apache Spark's resilient distributed datasets, and the log-centric architectures that process unbounded data.



Apache Spark and Resilient Distributed Datasets

Zaharia et al. (NSDI 2012), "*Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*," introduce the RDD: a read-only, partitioned collection of records that can be rebuilt from **lineage** — the sequence of deterministic transformations used to derive it — rather than from replicated copies.

Fault tolerance via lineage

On node failure, only the lost partitions are recomputed by replaying their transformation history — no cross-cluster replication write cost during normal operation, unlike Dynamo/HDFS-style approaches.

In-memory caching

`RDD.cache()/persist()` keep working sets resident in RAM (or spill to disk under pressure) across iterations, avoiding the repeated HDFS round-trips that penalize MapReduce iterative jobs.

Reported speedup

The original paper reports Spark running logistic regression and PageRank up to 20× faster than Hadoop MapReduce for iterative workloads on the same cluster.

Zaharia, M., Chowdhury, M., Das, T., et al. (2012). "Resilient Distributed Datasets..." Proc. 9th USENIX NSDI.

Spark's Execution Model and Library Stack

Transformations (map, filter, join, groupByKey) are lazy — they build a logical DAG of dependencies without computing anything. **Actions** (count, collect, saveAsTextFile) trigger the DAG scheduler to split the graph into stages at shuffle boundaries and submit tasks to the cluster.

Unified library stack (all built on the same Spark Core / RDD engine):

Spark SQL

Structured queries over DataFrames/
Datasets with a cost-based optimizer
(Catalyst).

Spark Streaming / Structured Streaming

Micro-batch (and later continuous)
processing of unbounded data as a
sequence of small RDD/DataFrame
batches.

MLlib

Distributed machine-learning algorithms
(classification, clustering, collaborative
filtering) operating directly on RDDs/
DataFrames.

GraphX

Graph-parallel computation (PageRank,
connected components) via a property-
graph abstraction atop RDDs.

Note the contrast with Hadoop: MapReduce required separate systems (Hive, Mahout, Storm) stitched together; Spark's DAG scheduler and shared RDD abstraction let one engine serve batch, SQL, streaming, ML, and graph workloads.

Apache Kafka and the Commit-Log Abstraction

Kreps, Narkhede, and Rao (LinkedIn, NetDB 2011), "*Kafka: A Distributed Messaging System for Log Processing*," reframe the publish-subscribe messaging problem around a **distributed, partitioned, replicated commit log** rather than a transient message queue.



- Partitioning:** A topic is split into ordered, immutable partitions; ordering is guaranteed only within a partition, enabling horizontal scale of both writes and reads.
- Consumer offsets:** Each consumer group tracks its own read position (offset) per partition — pub-sub with pull-based, replayable consumption rather than server-side per-consumer state.
- Durability:** Partitions are replicated across brokers; the log is retained for a configurable window, decoupling producer and consumer processing rates entirely.

Stream Processing Frameworks and Semantics

Apache Storm

Topology of spouts/bolts; tuple-at-a-time processing with low latency; "at-least-once" delivery by default (Nathan Marz, 2011).

Spark Streaming

Micro-batches unbounded input into small time-sliced RDDs, trading a small latency window for exactly-once semantics and unification with the batch API.

Apache Flink

True event-at-a-time dataflow engine with native event-time processing and watermarking (Carbone et al., 2015) — distinguishes event time from processing time explicitly.

Structured Streaming (Spark 2.x+)

Declarative, table-like streaming API; treats a stream as an unbounded, continuously appended table (Armbrust et al., 2018).

Delivery semantics matter architecturally: at-most-once (may lose data), at-least-once (may duplicate), and exactly-once (requires idempotent sinks or transactional writes, e.g., Kafka's transactional producer API, 2017) trade off latency, complexity, and correctness.

Lambda Architecture vs. Kappa Architecture

Lambda Architecture

Marz & Warren, Big Data (Manning, 2015)

Runs two parallel pipelines over the same source data: a batch layer (MapReduce/Spark, high latency, high accuracy) recomputes complete views; a speed layer (Storm/Spark Streaming) provides low-latency approximate views over recent data; a serving layer merges both for queries.

Trade-off: Cost: identical business logic must be implemented and kept consistent in two separate codebases/systems.

Kappa Architecture

Kreps, J. (2014), "Questioning the Lambda Architecture," O'Reilly Radar

Treats the immutable, replayable log (Kafka) as the single source of truth. All processing — real-time and "batch" — is expressed as stream processing; reprocessing means replaying the log from an earlier offset through an updated job, not maintaining a separate batch system.

Trade-off: Requires a stream-processing engine expressive and efficient enough to also serve historical reprocessing.

PART III

NoSQL & Storage Models

Why the CAP theorem forced a taxonomy of non-relational stores, and how each family resolves the trade-off.



NoSQL: Motivation and Taxonomy

Cattell (2011), "*Scalable SQL and NoSQL Data Stores*," SIGMOD Record, surveys systems that relax full ACID transactions and fixed schemas in exchange for horizontal scalability, higher throughput, and operational simplicity at scale.

Key-value stores

DynamoDB, Riak, Redis

Simplest model — opaque value retrieved by key; typically highest throughput and horizontal scalability.

Column-family stores

Bigtable, HBase, Cassandra

Rows with a variable, sparse set of columns grouped into families; efficient for wide, sparse datasets and range scans on row key.

Document stores

MongoDB, CouchDB

Self-describing semi-structured documents (JSON/BSON); schema-per-document, queryable on nested fields.

Graph databases

Neo4j, JanusGraph

Nodes and edges as first-class citizens; optimized for traversal-heavy queries (shortest path, recommendation) that are costly as SQL joins.

Common thread across all four families: schema flexibility and partition-tolerant horizontal scaling, purchased by relaxing some combination of consistency, joins, or transactional guarantees — formalized next by the CAP theorem.

The CAP Theorem and PACELC

Eric Brewer's 2000 PODC keynote conjecture, formally proved by Gilbert and Lynch (2002): a distributed system can provide at most two of three guarantees simultaneously **during a network partition**.

Consistency (C)

Every read receives the most recent write or an error — all nodes see the same data at the same time.

Availability (A)

Every request receives a (non-error) response — without the guarantee that it contains the most recent write.

Partition tolerance (P)

The system continues to operate despite an arbitrary number of dropped or delayed messages between nodes.

In practice, P is not optional at scale, so the real choice at partition time is **CP** (e.g., HBase, MongoDB by default) vs. **AP** (e.g., Cassandra, DynamoDB).

PACELC (Abadi, 2012) extends the picture: *if Partitioned, choose Availability or Consistency; Else (no partition), choose Latency or Consistency* — capturing that even absent failures, replicated systems still trade consistency for latency on every request.

Gilbert, S., & Lynch, N. (2002). "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services." ACM SIGACT News.

Dynamo and Bigtable — The AP/CP Reference Points

Dynamo (Amazon)

DeCandia et al., SOSP 2007

- AP system: prioritizes availability and partition tolerance over strict consistency.
- Consistent hashing distributes keys across nodes; a "preference list" replicates each key to N successor nodes.
- Sloppy quorum + hinted handoff keeps writes available during node failure; conflicting versions resolved via vector clocks, sometimes at the application layer.
- Directly inspired Cassandra (Facebook, 2008) and Riak.

Bigtable (Google)

Chang et al., OSDI 2006

- CP-leaning: strong consistency per row; built on GFS for storage and Chubby for locking/coordination.
- Sparse, distributed, persistent multi-dimensional sorted map: (row:string, column:string, timestamp:int64) → bytes.
- Rows sharded into tablets by row-key range; column families group related columns physically on disk.
- Directly inspired HBase (open-source) and influenced Cassandra's column-family data model.

Data Warehouses vs. Data Lakes

Data Warehouse

Schema-on-write: data is cleaned, transformed, and loaded into a predefined relational schema (star/snowflake) before storage — the classical ETL pattern (Kimball & Ross, The Data Warehouse Toolkit). Optimized for structured, well-understood, repeated analytical queries (OLAP).

Data Lake

Schema-on-read: raw data (structured, semi-structured, unstructured) is stored as-is in cheap distributed storage (HDFS, S3); schema/structure is imposed at query time. Favors exploratory analytics and heterogeneous data at the cost of upfront governance.

Convergence: the "lakehouse" pattern (Armbrust et al., 2021, Databricks/CIDR) layers transactional guarantees and schema enforcement (via formats such as Delta Lake, Apache Iceberg, Apache Hudi) on top of lake storage — targeting warehouse-grade reliability without the upfront ETL rigidity.

SYSTEMS IN PRODUCTION

Google Web Search Indexing

The original motivating use case for MapReduce (Dean & Ghemawat, 2004): rewriting the production indexing system as a sequence of five to ten MapReduce operations reportedly simplified the codebase substantially and made it easy to reason about fault tolerance and re-execution, compared to the earlier hand-parallelized version.

Amazon's Shopping Cart (Dynamo)

Dynamo was built specifically because a relational database could not meet Amazon's availability SLA for the "always writable" shopping cart during infrastructure or network failure — a business requirement to prefer availability over strict consistency, with conflict resolution deferred to application logic ("merge carts") at read time.

LinkedIn's Unified Data Pipeline

Kreps et al. (2011) describe replacing a tangle of point-to-point data feeds (batch file transfers, custom ETL) between systems (databases, Hadoop, search, monitoring) with Kafka as a single central log — every producer publishes once, every consumer (including Hadoop ingestion) reads independently at its own pace.

Key Takeaways

- Big data is a systems-scale problem, not a size threshold: it starts wherever single-machine architecture becomes the bottleneck.
- GFS/HDFS + MapReduce established durable, fault-tolerant batch processing over commodity clusters — at the cost of disk I/O between every stage.
- Spark's RDD/lineage abstraction removed that cost for iterative and interactive workloads, unifying batch, SQL, streaming, ML, and graph processing.
- NoSQL systems exist on a spectrum defined by the CAP theorem; the choice of store follows directly from the application's consistency/availability requirements.
- Kafka's commit-log abstraction turned streaming from a bolt-on feature into the architectural center (Kappa) of modern data platforms.

REQUIRED READING

Dean, J. & Ghemawat, S. (2004). MapReduce. OSDI.

Ghemawat, S. et al. (2003). The Google File System. SOSP.

Zaharia, M. et al. (2012). Resilient Distributed Datasets. NSDI.

Chang, F. et al. (2006). Bigtable. OSDI.

DeCandia, G. et al. (2007). Dynamo. SOSP.

Gilbert, S. & Lynch, N. (2002). Brewer's Conjecture. SIGACT News.

Cattell, R. (2011). Scalable SQL and NoSQL Data Stores. SIGMOD Record.

White, T. Hadoop: The Definitive Guide (O'Reilly).

Marz, N. & Warren, J. Big Data (Manning, 2015).